New issue

Efficient IIR filter bank design? #15

Closed

drewandre opened on Jul 17, 2020 Last edited by drewandre

Hi,

I was wondering if someone could point me in the right direction for designing an IIR-based filter bank. My goal is to recreate the MSGQ7's 7-band peak detection using the esp-dsp library.

I originally semi-successfully used this library's FFT function, but I couldn't get the resolution I needed in lower frequencies as I noted in this reddit post. I'm no expert in DSP, but a reddit user told me that a 44100fs/1024 point FFT yields a 20ms FFT frame window, and low-period frequencies (<200hz) would cause oscillations in the lower bins. This is exacerbated by spectral leakage.

As suggested in the reddit post, I moved to using this library's IIR filter. For each of the 7 MSGEQ7 bands (datasheet here) I calculated the coefficients using this handy online IIR biquad filter calculator. My "peak detector" logic for 63hz (which is just an average of all frequencies in a bandpass filter output) is as follows:

```
#define N_SAMPLES (2048)
#define N_SAMPLES_HALF (1024)

float *filter_output = (float *)calloc(N_SAMPLES_HALF, sizeof(float));
float *y_cf = (float *)calloc(N_SAMPLES, sizeof(float));

// x1 continuously updated from bluetooth a2dp i2s stream in my case
float *x1 = (float *)calloc(N_SAMPLES_HALF, sizeof(float));

float coeffs[5] = {
  0.0004485915993143372,
  0,
  -0.0004485915993143372,
  -1.9990222852850883,
  0.9991028168013714,
};

void calculate_average() {
  float w_lpf[5] = {0, 0};
```

```
  dsps_biquad_f32(x1, filter_output, N_SAMPLES_HALF, coeffs, w_lpf);

  for (int i = 0; i < N_SAMPLES_HALF; i++) {
    y_cf[i * 2 + 0] = filter_output[i];
    y_cf[i * 2 + 1] = 0;
  }

  dsps_fft2r_fc32(y_cf, N_SAMPLES_HALF);
  dsps_bit_rev_fc32(y_cf, N_SAMPLES_HALF);

  for (int i = 0; i < N_SAMPLES_HALF / 2; i++) {
    avg += (y_cf[i * 2 + 0] * y_cf[i * 2 + 0] + y_cf[i * 2 + 1] * y_cf[i * 2 + 1]) /
N_SAMPLES_HALF;
  }
  avg /= N_SAMPLES_HALF;
  printf("%f\n", avg);
}
```

My question is (and maybe **@dmitry1945** you could help me out): is there a more efficient way to do this? Ultimately to get 7 bands (or more), I would need to loop over this 7 times and calculate an IIR filter *and* FFT before averaging the FFT results. This seems horribly inefficient, but I just don't know enough about IIR filter bank design to know any better.

Alternatively, does anyone know how I could get better lower frequency resolution using the FFT alone? I really liked this solution (I had 1/3rd octave band and a-weighting logic running flawlessly) but I need low frequency resolution. I see that the max sampling rate of esp32 i2s is 48000 so I don't think adjusting this would help with frequency resolution. But is there anything else that I could adjust to get better lower frequency resolution in my FFT?

Any advice would be greatly appreciated! Thanks.

dmitry1945 on Jul 19, 2020 Collaborator

Hi **@drewandre**
For this purpose better if you will use IIR filter bank.
You can look to the esp-dsp/examples/iir.

There you will find how to generate a single IIR filter:

```
ret = dsps_biquad_gen_lpf_f32(coeffs_lpf, freq, qFactor);
```

The frequency is just: your_frequency/sample_frequency. In your case it will be 163/44100, and so on. You will have 7 filters.
The Q factor you have to play with this parameter.
In this example you can tune your filters.

At the output of your filters you make:

```
average[i] = K*average[i] + (1 - K)*absf(output[i]);
```

Where:
i - bank number from 0 to 6
K - smoothing coefficient 0.9..0.9999, 0.9999 - very smooth.
output[i] - output of i'th filter.
average[i] - result of your equalizer for i'th bank
absf - module operation.

Regards,
Dmitry

drewandre on Jul 21, 2020 Author

@dmitry1945 thanks, this was very helpful!

drewandre closed this as completed on Jul 21, 2020

generic-beat-detector on Mar 25, 2022 Last edited by **generic-beat-detector**

@dmitry1945

Hello sir. I applaud the effort behind this wonderful project!

I'd like some clarification please -- assuming the following for a single filter instance:

```
dsps_biquad_f32(x, y, N, coeffs_lpf, w_lpf);
```

How did the output `y` (array of floats) compute to `output[i]` (single float value)? I'm I missing something?

Regards.

drewandre on Mar 26, 2022 Author

It has been a long time since I've worked on this project but I pulled up the function where I averaged the filter output. Hopefully this helps.

```
void calculate_iir_filter_bank() {
  float avg, x;
  for (int i = 0; i < NUM_AUDIO_ANALYSIS_BANDS; i++) { // NUM_AUDIO_ANALYSIS_BANDS was 7 in my
case to match MSGEQ7
    avg = 0.0f;

    dsps_biquad_f32(buffer, filter_output, N_SAMPLES, filter_coefficients[i],
filter_delay_lines[i]);

    for (int t = 0; t < N_SAMPLES; t++)
    {
      x = fabs(filter_output[t]) * 4.0f;
      avg += x;
    }
    avg /= (float)N_SAMPLES;

    avg *= filters_amp_adjustments[i];

    if (avg > 1) {
      avg = 1;
    }

    averages[i] = (SMOOTHING_FACTOR * averages[i]) + ((1 - SMOOTHING_FACTOR) * avg);
  }
}
```

1

Add a comment

Write Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Reopen issue Comment

Please follow this repository's contributing guidelines.

Metadata

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



Open in GitHub Copilot app

Give feedback